

HeirRight Lead Engine — Cloud Scraper PRD



INTERNAL ONLY — DO NOT SHARE WITH CLIENT. This doc assumes the HeirRight deal is closed. It contains productized specs, sales-play notes, and architectural decisions that belong to Solvys. Client-facing scope lives on the shared HeirRight page.

Access posture: Public = no_access. Workspace inherit only. No explicit client user shares.



Product: Probate Lead Engine (PLE) | **Owner:** TP / Solvys | **Status:** Build-ready v1.0 | **Runtime Target:** Week 2 E2E, Week 1 Live Artifact
First deployment: HeirRight (Miami-Dade). Future deployments: any probate/estate operator with county-level public records access.

Product Positioning

PLE is a **productized cloud lead engine** for probate/heir-property operators. It scrapes public county records, enriches with skip trace, scores with Claude, and syncs to a CRM — end-to-end, cloud-hosted, client-owned IP.

Reusability: The core engine is client-agnostic. Per-deployment config = county list, source adapters enabled, CRM credentials, scoring prompt tuning. Ship new clients in < 1 week once v1 is live.

Solvys offer tier: Fits **SolvysOne+** — bespoke build, ~1–2 week implementation, \$6k–\$12k range depending on county count + data sources.

Problem (Generic)

Probate operators lose deal velocity because lead sourcing is:

- Manual: one human clicking through 5– 8 county portals daily.
- Unscored: every lead looks equal; reps waste calls on dead deals.
- Disconnected: data lands in spreadsheets, not CRM — no pipeline, no dedupe, no attribution.
- Untracked: no record of what was scraped when, or why a lead scored high.

PLE collapses this into a daily automated run with a ranked queue and full audit trail.

Proposal (Architecture)

Four layers, strict contracts between them:

1. **Ingestion** — Source adapters pull raw records on a schedule.
2. **Normalization** — Raw records → canonical **Lead** schema + dedupe.
3. **Enrichment + Scoring** — Skip trace fills contacts; Claude assigns score + reason codes.
4. **Sink + Artifact** — Writes to CRM, archives CSV, feeds live dashboard.

Decided infra (not open-ended):

- **Host:** Railway (Node worker + Postgres) for v1. Rationale: supports long-running headless browser, managed Postgres, simple deploys, cheap. Cloudflare Workers considered — rejected for v1 due to headless browser + >30s runtime needs.
- **Language:** TypeScript (Node 20+).
- **Headless browser:** Playwright (Chromium).
- **DB:** Postgres (Railway managed) with Prisma ORM.
- **Queue:** Postgres-backed (Graphile Worker) — no separate Redis for v1.
- **Secrets:** Railway variables + 1Password for dev.
- **Frontend (Live Artifact):** Next.js 14 app on Vercel, App Router, Tailwind + shadcn/ui.

- **Auth (artifact):** Clerk or a single shared password (Basic auth middleware) — decide at kickoff.
- **LLM:** Anthropic Claude Sonnet 4.5 for scoring; batch API where available.
- **CRM:** Close CRM REST API.
- **Observability:** Axiom for logs, BetterStack for uptime, Sentry for errors.

Full Product Specs

Canonical Data Model

```
// Lead: the deduplicated, scored unit that flows to CRM.
type Lead = {
  id: string // ULID
  sourceRefs: SourceRef[] // which source(s) + raw record IDs contributed
  owner: { name: string; aka?: string[] }
  property: {
    address: string
    city: string
    state: string
    zip: string
    parcelId: string // dedupe key #1
    county: string
    lat?: number; lng?: number
  }
  case?: {
    caseNumber: string // dedupe key #2
    caseType: "probate" | "tax_delinquent" | "foreclosure" | "other"
    filingDate: string // ISO date
    stage?: string
  }
  heirs: Contact[] // from skip trace + case doc
```

```

s
enrichment: {
  equityEstimate?: number
  assessedValue?: number
  lastSaleDate?: string
  skipTraceProvider?: string
  skipTraceRunAt?: string
}
score: {
  value: number // 0–100
  reasons: ScoreReason[]
  model: string // e.g., "claude-sonnet-4.5"
  scoredAt: string
} | null
sync: {
  crmLeadId?: string
  lastSyncedAt?: string
  syncStatus: "pending" | "synced" | "failed"
}
createdAt: string
updatedAt: string
}

type Contact = {
  role: "heir" | "executor" | "attorney" | "owner" | "other"
  name: string
  phones: string[]
  emails: string[]
  addresses: string[]
  confidence: number // 0–1 from skip trace
}

type ScoreReason = {
  code: string // e.g., "HIGH_EQUITY", "EARL
Y_PROBATE", "CONTACTABLE"
  weight: number

```

```

    note: string
  }

  type SourceRef = {
    source: SourceKey
    rawId: string
    fetchedAt: string
  }

  type SourceKey =
    | "clerk_of_courts"
    | "property_appraiser"
    | "probate_court"
    | "tax_collector"
    | "skip_trace"

```

Source Adapter Interface

Every source implements the same contract. One file per adapter. Max 300 lines.

```

export interface SourceAdapter {
  key: SourceKey
  fetch(input: {
    county: string
    dateRange: { start: string; end: string }
    cursor?: string
  }): Promise<{
    records: RawRecord[]
    nextCursor?: string
    fetchedAt: string
  }>
  healthcheck(): Promise<{ ok: boolean; reason?: string }>
}

```

Adapter rules:

- Stateless. No writes from the adapter.
- Respect robots.txt + ToS where possible. Flag any source requiring auth'd access.
- Exponential backoff on 429/5xx. Hard cap: 3 retries, then fail the run slice (not the whole run).
- Every fetch returns a deterministic `rawId` per record for dedupe.

Scoring Contract

- Input: a normalized Lead minus `score` field.
- Output: `{ value: 0-100, reasons: ScoreReason[] }`.
- Batched: up to 20 leads per Claude call.
- Prompt versioned in `/prompts/score-v{n}.md`. Bumping version is a deploy.
- Reason codes are an enum (locked). New codes require a schema PR.

CRM Sink Contract

- Upsert by `(parcelId, caseNumber)` composite key.
- Custom fields required in Close: `ple_score`, `ple_reasons`, `ple_source_keys`, `ple_parcel_id`, `ple_last_scored_at`.
- Bootstrap script creates missing custom fields on first run.
- Failed syncs write to `dead_letter` table + Axiom alert.

Auth + Access

- **Worker:** no public ingress. Internal API secured by HMAC-signed requests from the Next.js app.
- **Artifact frontend:** per-client password (env var `ARTIFACT_PASSWORD`) via middleware. Upgrade to Clerk when multi-tenant.
- **Admin routes:** separate password `ADMIN_PASSWORD` — Solvys-only.

Observability

- **Logs:** structured JSON, shipped to Axiom. Fields: `runId`, `source`, `county`, `leadCount`, `latencyMs`, `errorCode`.
- **Metrics:** leads-per-run, fill-rate %, score-distribution, sync-success %.
- **Alerts:** > 10% run failure, < 50 leads on a scheduled run, Claude API errors > 5%.
- **Uptime:** BetterStack monitor on the artifact URL + worker healthcheck endpoint.

Rate Limiting + Anti-Ban

- Per-source concurrency cap in env (`SOURCE_CONCURRENCY`).
- Residential proxy pool (Bright Data) behind a feature flag — on by default for Clerk of Courts.
- User-agent rotation from a short curated list. No spoofing headers beyond UA.
- Random jitter (250–1500ms) between page fetches.
- Circuit breaker: 3 consecutive 403s on a source → pause that source for 24h + alert.

Compliance Posture

- Scrape only publicly accessible records. No auth'd portals without explicit written client permission.
- Data retention: raw records 90 days, normalized leads indefinite.
- Contact data (from skip trace) purged from leads that are never actioned after 180 days.
- DNC/DNE suppression list honored pre-sync to CRM.

Live Artifact (Client-Facing Proof-of-Life)

- Branded per client. Dark mode, glass panels, gold accents (match client or Solvys house style).

- Password-gated subdomain.
- Live counter, live lead feed (last 20), run log, manual-trigger button, daily featured lead brief.
- v0 ships week 1 with one source, one county. Full source coverage by week 2.
- Slots into Lane D.

Repo Skeleton (VSCode Handoff)

```

probate-lead-engine/
├─ apps/
│   └─ worker/                                # Node worker (Railway)
│       └─ src/
│           └─ index.ts                        # entrypoint, cron regis
tration
│               └─ orchestrator.ts            # run lifecycle
│               └─ adapters/
│                   └─ clerk-of-courts.ts
│                   └─ property-appraiser.ts
│                   └─ probate-court.ts
│                   └─ tax-collector.ts
│                   └─ skip-trace.ts
│               └─ normalize/
│                   └─ dedupe.ts
│                   └─ canonicalize.ts
│               └─ score/
│                   └─ claude-client.ts
│                   └─ scorer.ts
│               └─ sinks/
│                   └─ close-crm.ts
│                   └─ csv-archive.ts
│               └─ lib/
│                   └─ logger.ts
│                   └─ retry.ts
│                   └─ proxy.ts

```



```

|   |   |   └─ api/                                # internal HMAC-signed A
PI for artifact
|   |   |       └─ runs.ts
|   |   |       └─ leads.ts
|   |   └─ prisma/
|   |       └─ schema.prisma
|   |   └─ prompts/
|   |       └─ score-v1.md
|   |   └─ tests/
|   |       └─ package.json
|   └─ artifact/                                # Next.js live dashboard
(Vercel)
|   └─ app/
|       └─ (public)/
|           └─ page.tsx                                # password gate
|       └─ (authed)/
|           └─ layout.tsx
|           └─ page.tsx                                # dashboard
|           └─ feed/
|           └─ runs/
|       └─ api/
|           └─ leads/route.ts
|           └─ trigger/route.ts
|   └─ components/
|   └─ lib/
|       └─ worker-client.ts                            # HMAC signer
|       └─ package.json
└─ packages/
    └─ types/                                # shared Lead/Run types
    └─ config/                                # per-deployment JSON co
nfigs
    └─ deployments/
    └─ heirright.json                            # county list, enabled s
ources, feature flags
└─ .github/workflows/
    └─ ci.yml

```

```
|   └─ deploy.yml
├─ docs/
|   └─ runbook.md
|   └─ add-a-source.md
|   └─ rollback.md
├─ .env.example
├─ pnpm-workspace.yaml
├─ turbo.json
└─ README.md
```

Hygiene: Max 300 lines per file. One adapter per file. Shared types in `packages/types`. Per-client config never hardcoded.

Env Vars (`.env.example`)

```
# Database
DATABASE_URL=

# Anthropic
ANTHROPIC_API_KEY=
CLAUDE_MODEL=claude-sonnet-4-5

# Close CRM
CLOSE_API_KEY=
CLOSE_PIPELINE_ID=

# Skip Trace (choose one)
SKIPTRACE_PROVIDER=batch
SKIPTRACE_API_KEY=

# Proxy
PROXY_URL=
PROXY_USER=
PROXY_PASS=
SOURCE_CONCURRENCY=3
```

```
# Artifact auth
ARTIFACT_PASSWORD=
ADMIN_PASSWORD=
WORKER_HMAC_SECRET=

# Observability
AXIOM_TOKEN=
SENTRY_DSN=

# Deployment
DEPLOYMENT_KEY=heirright
COUNTY_LIST=miami-dade
CRON_SCHEDULE="0 6 * * *"
```

Bootstrap Commands

```
pnpm install
pnpm --filter worker prisma migrate dev
pnpm --filter worker seed
pnpm dev          # runs worker + artifact in parallel via tur
bo
```

VSCode Handoff — Agent-Lane Prompts (4 agents × 2 prompts = 8)

Drop each prompt into a fresh Claude Code VSCode window. Lanes run in parallel. Each prompt is standalone.

Lane A — Ingestion Batch 1

A1 — Clerk of Courts adapter

- **Role:** Scraper engineer.

- **Objective:** Implement `apps/worker/src/adapters/clerk-of-courts.ts` to fetch probate filings for a given county + date range.
- **Constraints:** Max 300 lines. Playwright only when no JSON endpoint exists. Honor `SourceAdapter` interface. Exponential backoff, 3-retry cap.
- **Source-of-truth checklist:** Open `packages/types/src/Lead.ts`, `apps/worker/src/lib/retry.ts`, `apps/worker/src/lib/proxy.ts`. Verify Miami-Dade Clerk URL pattern by fetching the public search page and inspecting network calls.
- **Steps:** (1) Verify endpoint. (2) Define `RawRecord` for this source. (3) Implement paginated fetch with cursor. (4) Implement healthcheck. (5) Unit test with a captured HTML fixture.
- **Deliverables:** Adapter file, fixture, test, README note in `docs/add-a-source.md` if pattern differs from template.
- **Stop:** If the site requires login or blocks headless within 30 min of testing, stop and flag.

A2 — Property Appraiser adapter

- **Role:** Scraper engineer.
- **Objective:** Implement `apps/worker/src/adapters/property-appraiser.ts` returning parcel + assessed value for leads.
- **Constraints:** Same hygiene as A1. Prefer JSON endpoint if available.
- **Source-of-truth checklist:** Same as A1 plus inspect Miami-Dade PA site for documented JSON/XML feed.
- **Steps:** (1) Verify endpoint. (2) Schema for `RawRecord`. (3) Implement fetch by parcel or by address. (4) Healthcheck. (5) Test with real parcel ID.
- **Deliverables:** Adapter, test, sample output in `tests/fixtures/`.
- **Stop:** If no parcel API accessible, stop and document the HTML fallback path before writing the scraper.

Lane B — Ingestion Batch 2

B1 — Probate Court + Tax Collector adapters

- **Role:** Scraper engineer.
- **Objective:** Implement `probate-court.ts` and `tax-collector.ts` adapters.
- **Constraints:** Max 300 lines each. Share helpers via `apps/worker/src/lib/`.
- **Source-of-truth checklist:** Open Lane A adapters for pattern reference. Verify endpoints for Miami-Dade probate docket + Tax Collector delinquent list.
- **Steps:** (1) Verify both endpoints. (2) Implement both adapters. (3) Tests. (4) Healthcheck for each.
- **Deliverables:** Two adapters, two tests, updated `docs/add-a-source.md`.
- **Stop:** If either site requires CAPTCHA or login, stop and flag before writing scraper logic.

B2 — Skip Trace adapter + contact merge

- **Role:** Integrations engineer.
- **Objective:** Implement `apps/worker/src/adapters/skip-trace.ts` calling the selected provider, plus contact merge logic into `Lead.heirs`.
- **Constraints:** Provider-agnostic interface. Env-driven provider choice. Max 300 lines.
- **Source-of-truth checklist:** `packages/types/src/Lead.ts#Contact`, provider docs (env `SKIPTRACE_PROVIDER`).
- **Steps:** (1) Verify API contract. (2) Implement client. (3) Implement merge logic with confidence scoring. (4) Unit test merge with conflicting records.
- **Deliverables:** Adapter, merge module, tests.
- **Stop:** If API contract unclear, stop and request docs.

Lane C — Pipeline Core

C1 — Normalizer + dedupe + Prisma schema

- **Role:** Backend engineer.
- **Objective:** Implement `prisma/schema.prisma` for `Lead`, `Run`, `RawRecord`, `DeadLetter`. Implement `normalize/canonicalize.ts` and `normalize/dedupe.ts`.

- **Constraints:** Dedupe key = composite `(parcelId, caseNumber)` . Max 300 lines per file.
- **Source-of-truth checklist:** `packages/types` , canonical data model in this PRD.
- **Steps:** (1) Write Prisma schema. (2) Run migration. (3) Implement canonicalize to map any adapter output to `Lead` . (4) Implement dedupe with upsert semantics. (5) Tests covering merge of partial records from multiple sources.
- **Deliverables:** Schema, migration, two modules, tests.
- **Stop:** If data model gaps surface, update this PRD's data model section before coding.

C2 — Claude scorer + Close CRM sink

- **Role:** Backend engineer.
- **Objective:** Implement `score/scorer.ts` (batched Claude calls) and `sinks/close-crm.ts` (upsert + custom field bootstrap).
- **Constraints:** Score batches of 20. Prompt versioned in `prompts/score-v1.md` . Idempotent upsert by composite key.
- **Source-of-truth checklist:** Anthropic SDK docs, Close API docs, scoring contract in this PRD.
- **Steps:** (1) Write `score-v1.md` . (2) Implement batched scorer with retry + fallback to null score. (3) Implement Close upsert with custom field bootstrap. (4) Dead-letter path for sync failures. (5) Tests.
- **Deliverables:** Two modules, prompt file, tests.
- **Stop:** If Close custom fields cannot be created programmatically, stop and document manual bootstrap steps.

Lane D — Infra + Live Artifact

D1 — Worker infra: Railway deploy, cron, HMAC internal API, observability

- **Role:** Platform engineer.
- **Objective:** Wire orchestrator, Graphile Worker cron, internal HMAC-signed API routes for `runs` and `leads` . Ship to Railway. Ship logs to Axiom, errors to

Sentry.

- **Constraints:** No public ingress on worker. All API routes HMAC-verified.
- **Source-of-truth checklist:** Railway docs, Graphile Worker docs, Axiom SDK.
- **Steps:** (1) Orchestrator with run lifecycle. (2) Cron registration. (3) HMAC API. (4) Axiom + Sentry. (5) Railway deploy + healthcheck endpoint. (6) CI workflow.
- **Deliverables:** Working deployed worker, deploy URL, runbook entry.
- **Stop:** If Railway free tier insufficient for Playwright, bump plan and note cost in runbook.

D2 — Live Artifact: Next.js dashboard on Vercel

- **Role:** Frontend engineer.
- **Objective:** Build `apps/artifact` — password-gated dashboard with counter, live feed, run log, manual trigger, featured lead card. Deploy to Vercel.
- **Constraints:** Tailwind + shadcn/ui, App Router, dark mode default. Max 300 lines per component. Password middleware on all authed routes. Manual trigger hits worker via HMAC.
- **Source-of-truth checklist:** Next.js App Router docs, shadcn, `apps/worker/src/api/*` for contract.
- **Steps:** (1) Scaffold app. (2) Password middleware. (3) Dashboard + feed + run log pages. (4) Manual trigger button. (5) Featured lead card with daily brief. (6) Deploy to Vercel + custom subdomain. (7) Branding pass (client palette).
- **Deliverables:** Deployed artifact URL, screenshot in PR, branding checklist.
- **Stop:** If HMAC signing contract with worker is unclear, stop and align with D1.

Acceptance Criteria (Rollup)

- ☐ Daily cron completes full Miami-Dade sweep in < 30 min.
- ☐ ≥ 95% of leads have owner, address, parcel ID, case type.
- ☐ ≥ 70% of leads have a contact method post skip trace.

- ☐ Every lead in Close has a score + ≥ 1 reason code.
- ☐ Zero dupes on `(parcel_id, case_number)`.
- ☐ Live artifact loads < 2s, shows real last-24h leads.
- ☐ Manual trigger completes end-to-end in < 10 min for single county/day.
- ☐ One scraper failing does not block others.
- ☐ All deploys tagged; rollback in < 5 min.

Failure Modes + Rollback

- DOM change → 0 leads → Axiom alert + last-known-good fallback.
- Rate limit → backoff + checkpoint resume.
- Claude outage → queue `score=null pending=true`, reprocess next run.
- Close outage → write to CSV archive + dead-letter table, manual import path documented.
- Bad deploy → `railway rollback <tag>` + `vercel rollback` (< 5 min).

Payment + Ownership (Internal)

- First deployment (HeirRight): \$6,250, rolled into engagement on HeirRight Real Estate.
- Future deployments productized at \$8k–\$12k depending on source + county count.
- Per-deployment passthroughs: Railway (~\$20–\$50/mo), Vercel (free–\$20/mo), Anthropic (~\$0.05–\$0.15/lead scored), skip trace (provider rates), proxy (~\$50–\$200/mo).
- Code and IP transfer to client on engagement close via private repo ownership transfer or fork rights — Solvys retains the core engine rights for other deployments.



Sales Play Notes (Internal, never expose)

- **Anchor Advantage:** Live artifact in week 1 anchors the \$6,250 as "already working, already yours."
- **Authority by Analysis:** Claude-written daily lead brief demonstrates depth vs commodity scrapers.
- **Social Proof Injection:** "Same engine pattern we run for other ops" (never name other clients).
- **Mirror Trap:** Frame outputs in client's vocabulary ("heir deals," "probate filings") not ours.
- **Choice Illusion:** Offer 3 post-v1 tiers — (a) maintenance only, (b) new county add-on, (c) doc-prep + outreach automation bolt-on.



Open Items (Convert to Verification Steps at Kickoff)

- Skip trace provider lock-in (BatchSkipTracing vs IDI vs TLO — price/coverage test with 100 leads).
- Auth choice for artifact v1 (shared password now, Clerk at v1.1).
- Branding palette confirmation per deployment.
- CRM pipeline IDs + custom field naming convention approval.
- Document-prep template library (consumed by downstream workstream, not this build).